I. Identity & Access Management (IAM) Module

---

1. Overview

The Identity & Access Management (IAM) module of WellTrack is a core security component responsible for authentication, authorization, auditability, and secure user lifecycle management.

It ensures that only authenticated and authorized users can access system resources based on their assigned roles, while maintaining a complete audit trail for compliance and security monitoring.

The IAM module follows industry-standard security practices, including:

- Stateless authentication using JWT (JSON Web Tokens)
- Role-Based Access Control (RBAC)
- Strong password encryption using BCrypt
- Immutable audit logging
- Centralized exception handling
- Extensive automated testing (unit, controller, security)

---

2. Objectives

The IAM module is designed to achieve the following objectives:

- Provide secure authentication for all system users
- Enforce least-privilege access using role-based authorization
- Prevent unauthorized access and privilege escalation
- Support auditability and compliance requirements
- Enable scalable and stateless authentication for distributed microservices
- Ensure high maintainability through clean architecture and test coverage

---

3. Supported User Roles

The system defines the following roles:

Role	Description

EMPLOYEE	Participates in wellness programs, tracks goals
HR	Creates and manages wellness programs
COACH	Manages activities and attendance
COMPLIANCE	Reviews compliance records and audits
ADMIN	Full system access and user management

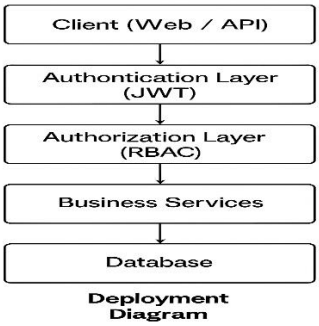
Each role is mapped to explicit permissions enforced at API method level.

---

4. Architecture Design

4.1 IAM Architectural Placement

Figure 4.1 – Authentication and Authorization Layered Architecture



Description:
This diagram illustrates the high-level placement of the IAM module. All incoming requests from web or API clients first pass through JWT-based authentication, followed by RBAC authorization, before accessing business services and database resources.

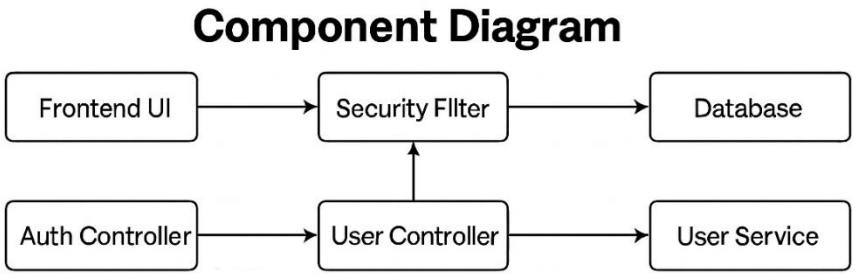
4.2 Design Principles

- Stateless Authentication – No server-side sessions
- Single Responsibility – IAM isolated from business logic
- Defense-in-Depth – JWT + RBAC + method-level security
- Auditability – Every critical security action is logged

---

5. Component Design

Figure 5.1 – Component Diagram



Description:
The component diagram shows how authentication, authorization, and user management components interact. The Security Filter ensures requests are validated before reaching controllers and services.

---

6. Key Features

6.1 Authentication (JWT-Based)

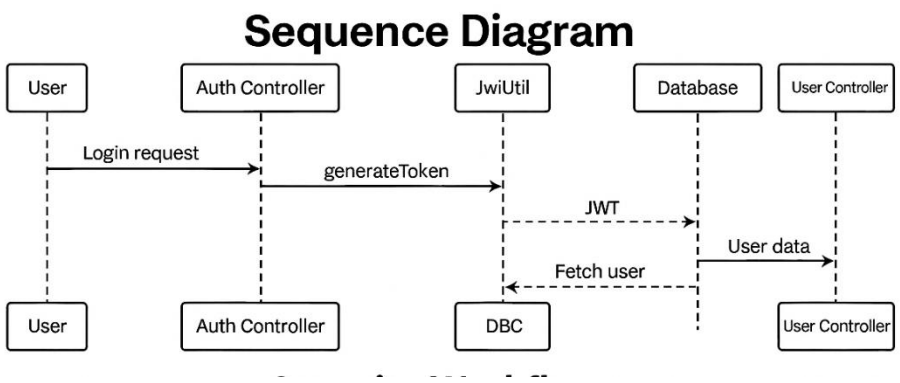
- Users authenticate via:
- POST /api/auth/login
- Upon successful authentication, a JWT token is generated
- The token is sent in subsequent requests using:
- Authorization: Bearer <JWT_TOKEN>

Why JWT?

- Stateless (no session storage)
- Ideal for microservices
- Horizontally scalable
- Industry standard (OAuth2, OpenID Connect)

6.2 Authentication Sequence

Figure 6.1 – Sequence Diagram (JWT Authentication Flow)



Description:
This sequence diagram explains the login flow, token generation, and retrieval of user information from the database during authentication.

6.3 Password Security

- Passwords are never stored in plain text
- Encrypted using BCrypt hashing
- Automatic salt generation
- Resistant to rainbow-table attacks

Example Workflow:

Plain Password → BCrypt Hash → Database

6.4 Role-Based Access Control (RBAC)

Authorization is enforced using Spring Security method-level annotations:

@PreAuthorize("hasRole('ADMIN')")

``

RBAC Access Matrix

API	ADMIN	HR	EMPLOYEE
Create User	✅	❌	❌
View All Users	✅	✅	❌
View Own Profile	✅	✅	✅

Update User	✅	❌	❌
Delete User	✅	❌	❌

✓ Enforced at method level
✓ Prevents accidental privilege leaks
✓ Easy to audit and maintain

---

7. Entity Design

7.1 User Entity

Purpose: Represents an authenticated system user.

Attributes:

- userId (Primary Key)
- name
- email (unique)
- password (encrypted)
- role
- phone
- status (ACTIVE / INACTIVE)

7.2 AuditLog Entity

Purpose: Tracks security-sensitive actions.

Captured Events:

- User creation
- User updates
- Role changes
- Account status changes
- Deletions

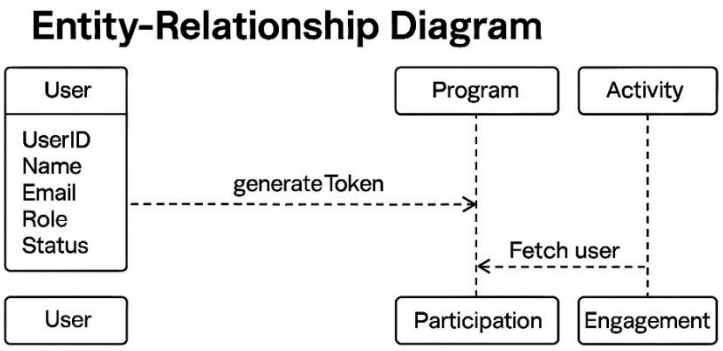
Attributes:

- auditId

- userId
- action
- resource
- timestamp

✓ Immutable
✓ Compliance-friendly
✓ Tamper-resistant

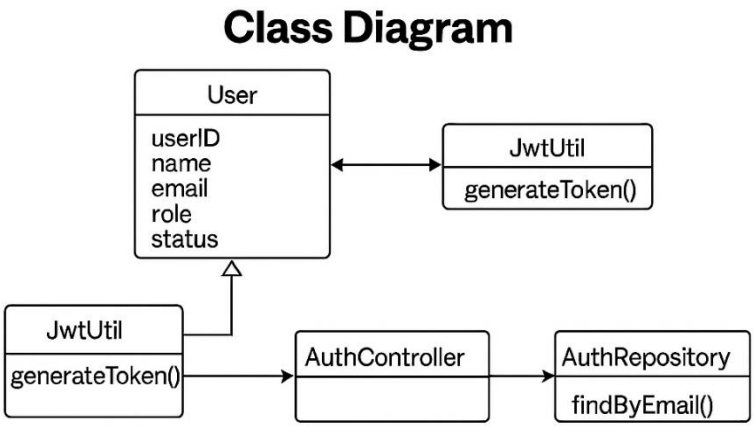
Figure 7.1 – Entity Relationship (ER) Diagram



---

8. Class Design

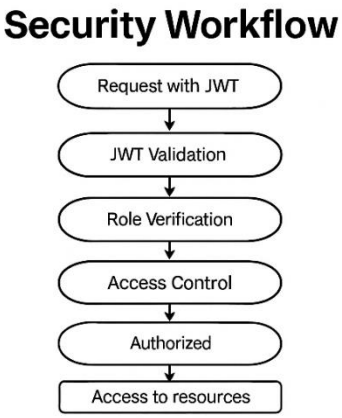
Figure 8.1 – Class Diagram



Description:
This class diagram illustrates the static structure of the IAM module, highlighting domain entities, controller logic, and authentication utilities.

9. Security Workflow

Figure 9.1 – Security Workflow Diagram



Workflow Steps:

1. User requests login
2. Credentials validated
3. JWT generated and returned
4. Client includes JWT in request header
5. JWT filter validates token
6. RBAC rules evaluated
7. Request allowed or denied

---

10. API Endpoints (IAM)

Authentication APIs

Method	Endpoint	Description
POST	/api/auth/login	Authenticate user and generate JWT

User Management APIs

Method	Endpoint	Access
POST	/api/users	ADMIN
GET	/api/users	ADMIN, HR

GET	/api/users/{id}	ADMIN, HR, EMPLOYEE
PUT	/api/users/{id}	ADMIN
PATCH	/api/users/{id}/activate	ADMIN
PATCH	/api/users/{id}/deactivate	ADMIN
DELETE	/api/users/{id}	ADMIN

All APIs are fully documented using Swagger (OpenAPI).

---

11. Exception Handling

- Centralized exception handling using @RestControllerAdvice
- Prevents sensitive error leakage
- Standardized error responses
- Improves maintainability and debugging

---

12. Logging & Monitoring

- Uses SLF4J for logging
- Logs:
    - Authentication attempts
    - Security violations
    - System errors
- Designed for centralized logging tools (ELK, Splunk)

---

13. Testing Strategy (Industry Practice)

13.1 Test Types Implemented

Test Type	Tool
Unit Tests	JUnit 5
Mocking	Mockito

Controller Tests	MockMvc
Security Tests	Spring Security Test
JWT Filter Tests	Mockito
RBAC Validation	@WithMockUser

13.2 Testing Benefits

- Early defect detection
- Security validation
- Regression prevention
- CI/CD compatibility

---

14. Compliance & Best-Practices Alignment

✓ OWASP secure password practices
✓ Stateless authentication
✓ Least-privilege principle
✓ Auditability
✓ Test-driven development
✓ Clean architecture

---

15. Scalability & Future Enhancements

- OAuth 2.0 / OpenID Connect integration
- Enterprise Single Sign-On (SSO)
- Multi-Factor Authentication (MFA)
- Token refresh and rotation
- Fine-grained permission model

---

16. Summary

The Identity & Access Management (IAM) module of WellTrack provides a robust, scalable, and secure foundation for the platform. By leveraging JWT authentication, RBAC, encrypted credentials, audit logging, and comprehensive testing, the system

meets modern enterprise security standards and is suitable for production-level deployments.